

第12课 存储系统 -part1

北京交通大学软件学院

严禁复制





01 复制 存储系统的重要性

为何存储选型至关重要？

- 存储选型对系统性能、成本、可扩展性、可靠性的深远影响
- Google设计BigTable的案例：
 - 可扩展性：Scale-up增强单机性能，Scale-out（横向扩展）。
 - 成本：商业数据库的授权费用和高端硬件成本巨大。
 - 数据模型灵活性：数据类型多样，结构化数据，如网页内容、用户行为日志等。
 - 性能和吞吐量：极高的读写吞吐量和低延迟的数据访问。
 - 运维复杂性：具备错误自动恢复的能力。
- 一个好的存储决策是成功架构的基石
 - Google的bigtable支持网页，地图，youtube用户数据，gmail数据
 - Bigtable: A Distributed Storage System for Structured Data
 - <https://static.googleusercontent.com/media/research.google.com/zh-CN//archive/bigtable-osdi06.pdf>

架构师在存储选型中的角色与责任

- 理解业务需求，并将其转化为技术需求
- 评估不同存储方案的优劣与权衡 (Trade-offs)
- 制定存储策略，并推动其落地
- 对最终选型负责

严禁复制

本次课程目标与议程

- 目标：
 - 掌握存储选型的核心考虑因素
 - 理解不同存储技术的适用场景
 - 培养基于场景进行存储选型的能力
- 议程概览：
 - 存储选型概览
 - 核心考量因素深度剖析
 - 综合选型与案例分析
 - 总结与展望

严禁复制



02 复制 存储系统分类

存储系统分类

- 定义：用于持久化存储和管理数据的软硬件系统
- 存储系统分类：
 - 按数据模型：关系型、键值、文档、列式、图、对象存储
 - 按部署模式：本地部署、云存储、混合云

严禁复制

关系型数据库

- 1970年 Edgar F. Codd "A Relational Model of Data for Large Shared Data Banks"
- Oracle 1979年: 将关系数据库商业化
 - ACID 特性 (Atomicity, Consistency, Isolation, Durability)
 - 保证了事务的可靠性, 成为企业级应用的核心
- MySQL (1995):
 - 以其易用性和高性能在 Web 应用中迅速普及
- PostgreSQL (起源于 Berkeley 的 Ingres 项目, 1996年正式命名):
 - 以其强大的功能、可扩展性和对 SQL 标准的严格遵守而闻名

键值数据库 key-value

- 早期形式:
 - 哈希表 (Hash Tables): 内存中的基本数据结构, 是键值存储最简单的形式。
 - Berkeley DB (BDB) (1990s初): 一个嵌入式的键值存储库, 被广泛使用。
- Web 时代的驱动 (2000s中期 至今):
 - Memcached (2003):
 - 高性能分布式内存对象缓存系统, 标志着大规模分布式缓存的开始。
 - (2007): "Dynamo: Amazon's Highly Available Key-value Store":
 - 高可用、高可扩展的分布式键值存储系统,
 - AP (可用性和分区容错性) 而非强一致性 (牺牲了C), 引入“最终一致性”。
 - Redis (2009):
 - 支持多种数据类型 (字符串、哈希、列表、集合、有序集合等) 。
- 重大影响:
 - 极高读写性能
 - 打破关系数据的垄断地位

文档数据库 (Document Databases)

- 起源与动机 (2000s中期):
 - XML 数据库的尝试: 在 JSON 流行之前, 基于 XML 。
 - 直接存储 (如 JSON/BSON) , 避免对象到关系表的映射, 简化了开发。
 - 灵活的模式:
- 关键发展:
 - CouchDB (2005): 最早的文档的数据库,
 - 使用 JSON 存储数据, 通过 MapReduce 进行查询。
 - MongoDB (2009): 最流行的文档数据库。
 - BSON (Binary JSON) ,
 - 丰富的查询语言、索引功能和易于扩展的架构。
- 重大影响:
 - 极大地简化后端开发。
 - 推动了 JSON 作为数据交换格式的普及。

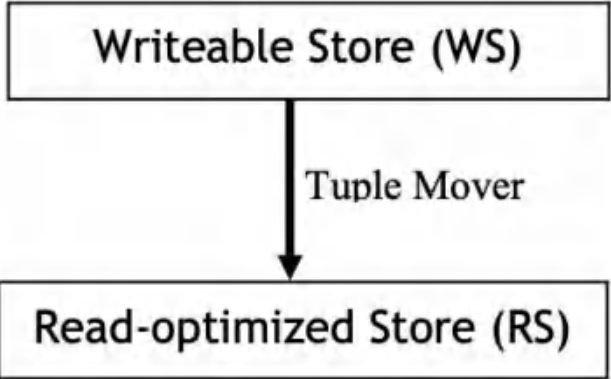
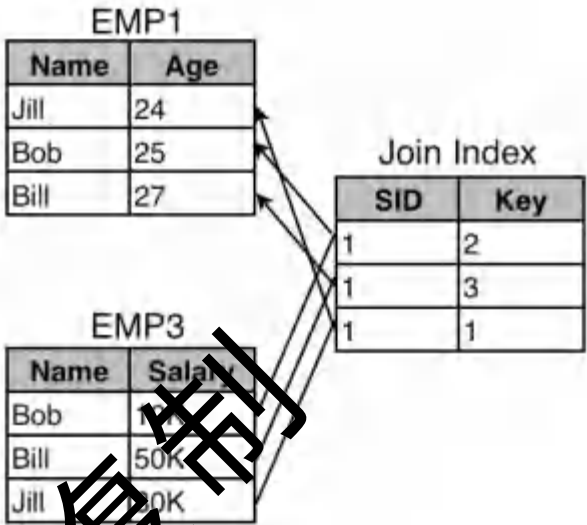
	JSON	BSON
存储格式	文本	二进制
人类可读性	高	低 (需要工具)
数据类型	有限 (字符串、数字等 6 种)	丰富 (JSON 类型 + ObjectId, Date, Binary, Int64 等更多)
大小	对于复杂/数字/二进制数据通常更大	对于复杂/数字/二进制数据通常更小
解析速度	较慢	较快
典型用途	网络数据交换、配置文件	数据库存储 (如 MongoDB)、内部数据序列化
关系	-	是 JSON 的超集

列式数据库 (1 of 3)

- 动机:
 - 传统行式数据库在进行数据分析（OLAP）查询时效率低下，需要扫描整行数据。
 - 列式存储只读取需要的列，大大提高分析查询性能和数据压缩率。
- C-Store 论文 (2005): "C-Store: A Column-oriented DBMS"
 - <https://people.csail.mit.edu/tdanford/6830papers/stonebraker-cstore.pdf>
- 商业和开源产品的涌现 (2000s后期 至今):
 - Vertica (源于C-Store),
 - Amazon Redshift,
 - Google BigQuery (远不止列式存储)
 - ClickHouse,
 - Apache Druid。
- 分析型列存:
 - 彻底改变了数据仓库和商业智能领域，使得对海量数据进行快速复杂的分析成为可能。

列式存储数据库 (2 of 3)

- 列式数据布局：
- 数据压缩
- 晚期物化
- 投影 和排序：
 - 不存储传统的表，而是存储表的“投影”
 - 一个投影包含表中的部分列，并且这些列以特定的顺序排序。
- 写优化存储和读优化存储
- 连接优化 (Join Optimization):
 - 排序投影，位图连接，hash join



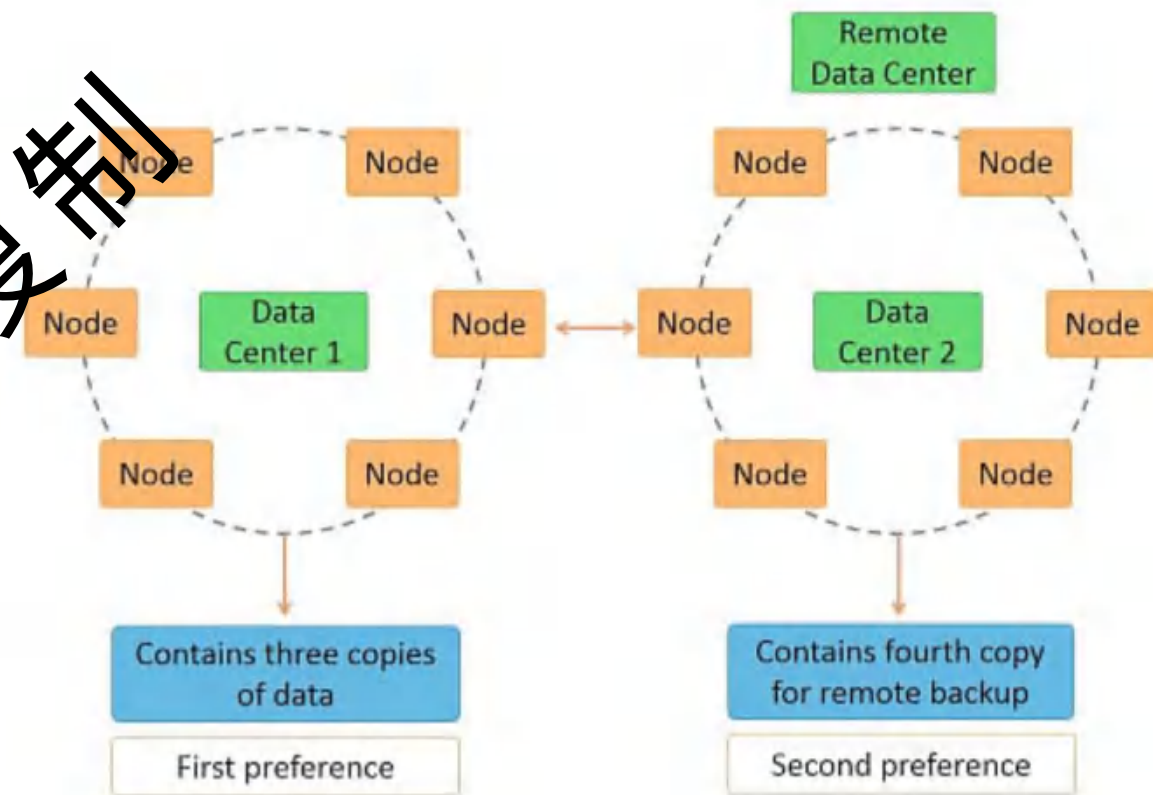
join性能对比

Query	C-Store	Row Store	Column Store
Q1	0.03	6.80	2.24
Q2	0.36	1.09	0.83
Q3	4.90	93.26	29.54
Q4	2.09	722.90	22.23
Q5	0.31	116.56	0.93
Q6	8.50	652.90	32.83
Q7	2.54	265.80	33.24

列式数据库（存储）

- 论文 (2006): "Bigtable: A Distributed Storage System for Structured Data":
 - 稀疏的、分布式的、持久化的多维排序映射表,
 - 专为管理PB级结构化数据而设计。
 - 不是传统意义上的列式数据库, 按列族组织。
- Apache HBase (2007开始):
 - 开源的、受 Bigtable 启发的分布式列式数据库, 构建在 HDFS 之上。
- Apache Cassandra (2008 Facebook开源):
 - 结合了 Amazon Dynamo 的分布式设计和 Google Bigtable 的数据模型,
 - 是一个高可用、高可扩展的宽列存储。

Cassandra



图数据库 (Graph Databases)

- 动机:

- 关系数据库在处理高度连接的数据（如社交网络、推荐系统、欺诈检测）时，需要大量的 JOIN 操作，性能会急剧下降。
- 图数据库专门优化节点、边和属性的存储与查询。

- 原生图数据库的兴起 (2000s后期 至今):

- Neo4j (2007首次发布, 商业公司2000年成立): 使用属性图模型和 Cypher 查询语言。
- JanusGraph (开源, 构建在其他存储之上如HBase, Cassandra),
- Amazon Neptune
- Azure Cosmos DB (支持图API)。

- 重大影响:

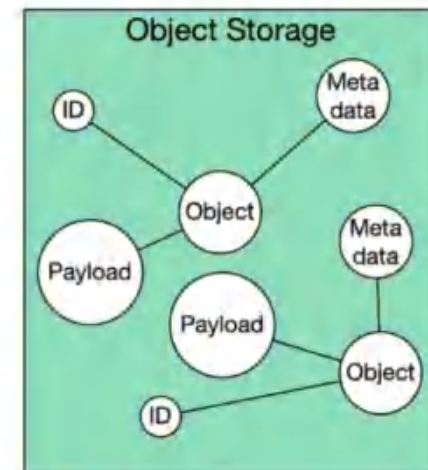
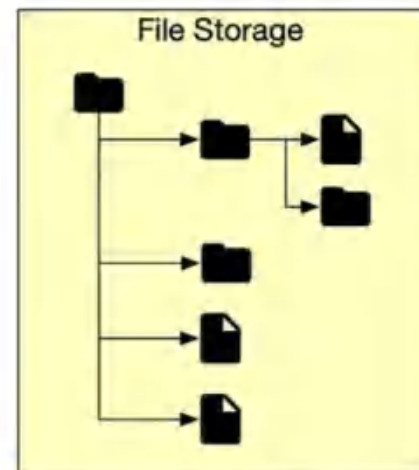
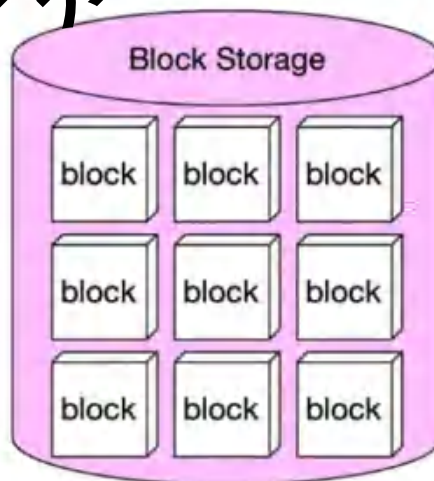
- 为处理复杂关系和网络数据提供了高效的工具,
- 在社交网络分析、推荐引擎、知识图谱、路径查找、金融风控等领域有广泛应用。



对象存储 (Object Storage)

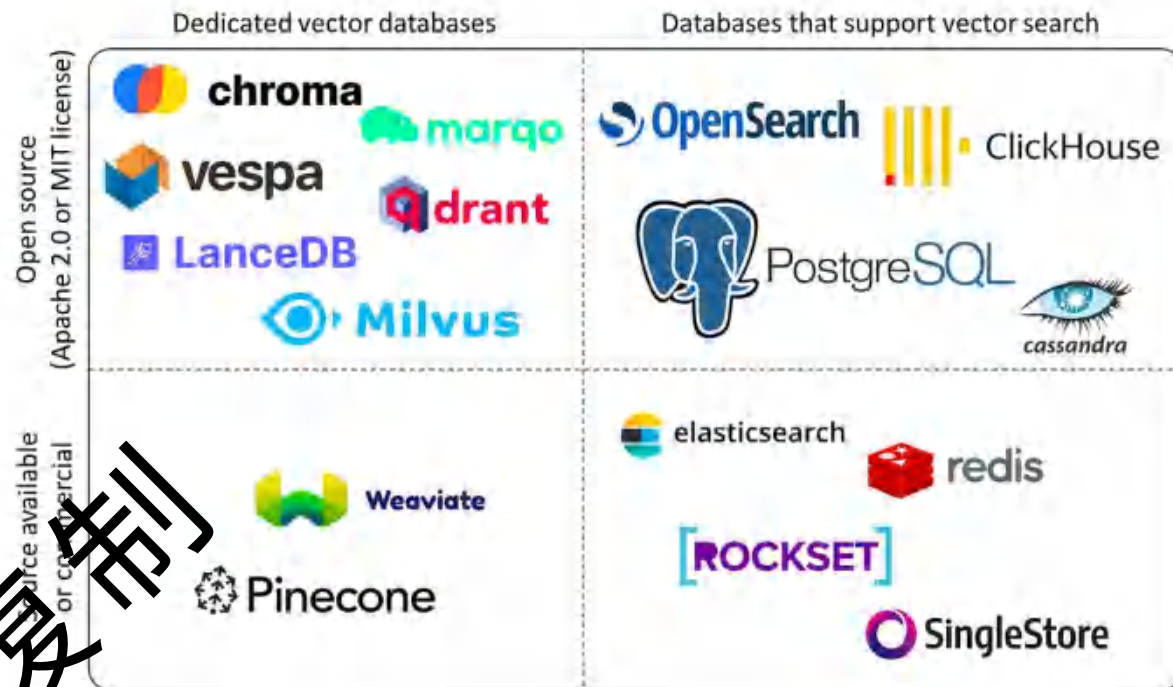
- 动机:
 - 海量的非结构化数据（如图片、视频、备份文件、日志归档）。
 - 对象存储提供了一个扁平的命名空间（通常是桶/对象键）。
 - 通过HTTP API访问，具有高持久性和近乎无限的扩展性。
- 关键发展:
 - Amazon S3 (Simple Storage Service) (2006):
 - 这是一个革命性的产品，定义了云对象存储的标准。
 - 其API（如GET, PUT, DELETE）成为行业标准。
 - Azure Blob Storage,
 - Google Cloud Storage 。
 - 开源对象存储系统: Ceph, MinIO。
- 重大影响:
 - 成为云上存储非结构化数据的事实标准。
 - 极大地简化了大规模数据存储和归档的管理。

严禁复制



向量数据库

- 动机：
 - 高维空间中进行**高效相似性搜索**的需求
- 驱动力：
 - 非结构化数据的爆炸式增长，语义理解的需求
- 传统数据库：
 - 不支持直接计算向量间的距离
 - 对于高维向量数据，传统数据库的索引（如B+树）效率极低
 - 传统的空间索引结构（如k-d tree, R-tree）在维度非常高
- 发展历史
 - 早期探索：1995–2005
 - 近似最近邻（ANN）2000–2015
 - 嵌入技术的爆发（2010–今）
 - 专用数据库（2019–）：Milvus, Weaviate, Pinecone, Qdrant
 - 现有db的扩展：PostgresSql-pgvector, ES的k-NN插件



数据存储的发展趋势

- Codd 的关系模型论文 (1970): 奠定了现代数据库理论的基础。
- SQL 的标准化和普及 (1980s–1990s):
- 互联网和 Web 2.0 的爆发 (2000s): 高并发、高可扩展性、灵活模式存储的需求, 推动了 NoSQL 运动。
- Google 的 MapReduce (2004), Bigtable (2006) 和 Amazon 的 Dynamo (2007) 论文:
 - 大规模分布式系统的设计思想, 对 NoSQL 和大数据技术产生了深远影响。
- NoSQL 运动 (2000s后期至今): 各种非关系型数据库——“最适合的工具做最适合的事”。
- 云计算的普及 (2000s后期至今): 各种数据库服务易于获取和使用, 加速了新存储技术的采纳。
- 大数据时代的到来: 海量数据存储、处理和分析, 进一步推动了列式数据库、对象存储和分布式文件系统。
- LLM时代, 进一步推动向量数据库的发展。



03 复制 存储系统选型

存储选型的挑战与常见误区

- **挑战：**

技术选型多样性带来的选择困难

业务需求多变且难以预测

性能、成本、一致性之间的永恒权衡

- **常见误区：**

追求“最好”而非“最合适”的技术

忽视长期运维成本和复杂度

盲目跟风，不深入分析业务场景

严禁复制

选型流程与方法论概述

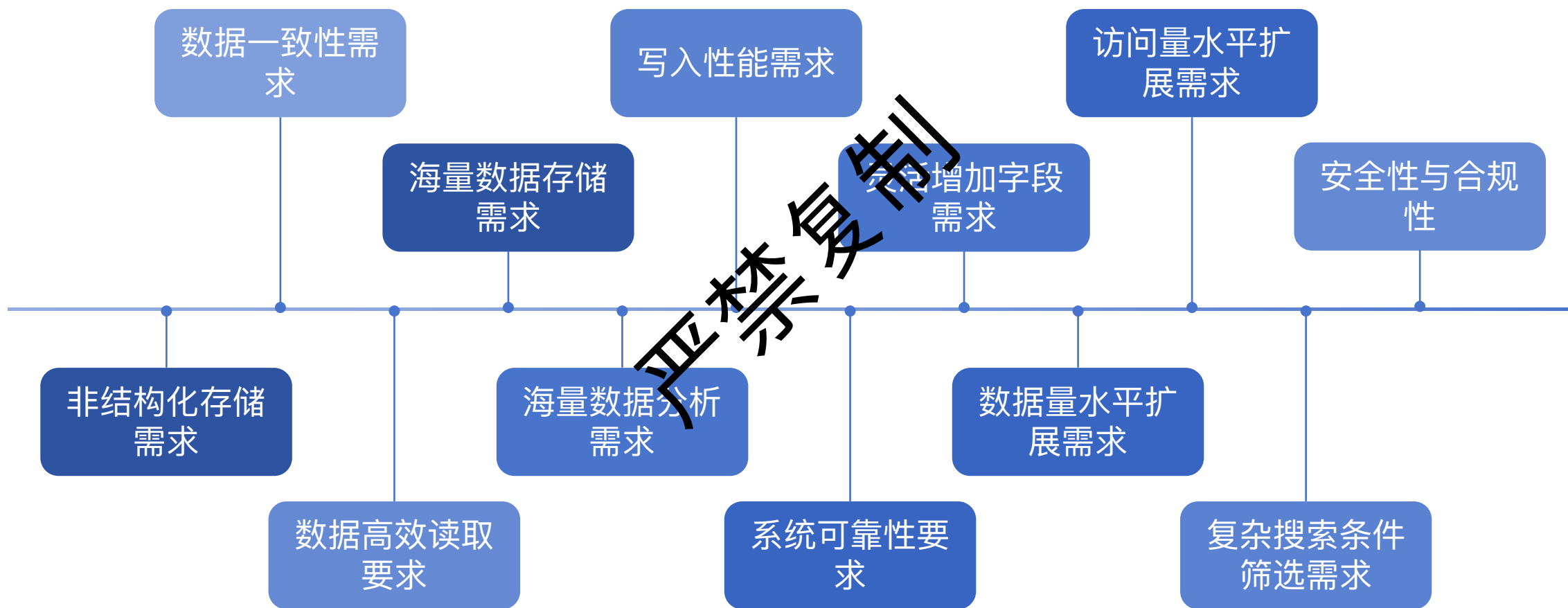


“没有银弹”：理解业务需求是前提

- 任何存储技术都有其特定的适用场景和局限性。
- 强调架构师的核心任务是**深刻理解业务**，而非仅仅堆砌技术。
- 不同业务场景对存储的需求千差万别（举例：电商秒杀 vs. 订单管理）

特性	电商秒杀 (Flash Sales)	订单管理 (Order Management)
核心挑战	瞬时超高并发写、库存准确性	海量历史数据管理、复杂查询、数据一致性
并发量	极高，尖峰特征明显	中高，相对平稳
延迟要求	极低，毫秒级	合理，秒级可接受
一致性	库存强一致性至关重要，其他可适当放宽	核心业务数据（订单状态、金额等）强一致性
可用性	极高，不容有失	高，需保证业务连续性
扩展性	需具备极强的弹性伸缩能力	需具备良好的水平扩展能力以应对数据增长
数据生命周期	短暂的热点数据	长期存储的事务数据和历史数据
读写侧重	瞬时大量读（商品页）和极高写（库存、订单）	读写相对均衡，查询复杂
存储技术倾向	内存数据库、高性能分布式数据库、消息队列	关系型数据库为主，辅以NoSQL、数据仓库

12个关键因素





因素制： 存储的数据结构

应用场景

- 定义：
 - 存储非固定结构或半结构化数据，如媒体文件、文档、日志等。
- 典型场景：
 - 用户上传的图片、视频、音频（社交媒体、内容平台）
 - 医疗影像数据 (DICOM)
 - 文档管理系统 (PDF, Office文档)
 - 备份与归档

严禁复制

深化的问题

数据结构：

高度结构化的（如关系表）、
半结构化的（JSON、XML）、
非结构化的（文本、日志），
还是二进制大对象（图片、音
视频文件）？

数据关系：

数据之间是否存在复杂的关系？
（如：一对多、多对多、图谱
关系）

Schema 灵活性：

Schema 是固定的，
需要频繁变更、
根本不需要 Schema
(Schema-on-read)？

进一步深化的问题



对象size大小、
数量？

大量小文件
少量大文件



是否结构化？

高度结构化的、
半结构化的、
非结构化的、
二进制大对象。



Schema 灵活性？

Schema 是固定的？
需要频繁变更？
no schema?



如何与计算服
务高效集成？

CDN、AI处理



存储成本如何
优化？

文件压缩、文件删
除

能力细分与相关数据库/技术

- 能力细分：
 - 大对象存储与分发
 - 元数据管理与检索
 - 生命周期管理（分层存储）
 - 版本控制
- 相关技术/数据库：
 - 对象存储服务：
 - Amazon S3, Azure Blob Storage, Google Cloud Storage, MinIO
 - NoSQL数据库（文档存储）：
 - MongoDB (GridFS), Couchbase
 - 分布式文件系统：
 - HDFS（适用于配合计算框架）

禁止复制

S3如何实现大对象的存储和分发

- 1. Multipart Upload (分段上传):
 - 大对象（通常大于100MB，最大可达5TB）
 - 本地分割成多个part独立上传。
- 2. 分布式存储架构:
 - 数据冗余：自动复制到多个服务器
 - 横向扩展：打散并分布到成千上万的服务器上
 - 元数据管理：对象的name, size, type, 权限
- 3. 数据分发 (Distribution):
 - 从边缘节点下载
 - Amazon CloudFront (CDN 集成)

S3数据读取过程





因素控制：
数据一致性

一致性的4种定义

定义：

强一致性 (Strong Consistency / Linearizability):

所有读操作都能看到写操作实时更新的数据。

顺序一致性 (Sequential Consistency):

所有进程以相同顺序观察到所有操作。

因果一致性 (Causal Consistency):

有因果关系的写操作对所有进程的顺序一致。

最终一致性 (Eventual Consistency):

如果没有新的更新，最终所有副本都会达到一致状态

要点：

操作的时间

操作的顺序

典型场景

强一致性：

金融交易、银行账户、
订单库存、
不允许读到旧数据或
脏数据

最终一致性：

社交网络动态、用户
点赞数、商品评论
(允许短暂不一致，
最终会同步)

因果一致性：

在线文档协作编辑

顺序一致性：

银行转账；
分布式锁操作；
全局配置修改；

典型场景：

相关数据库

01

关系型数据库 (默认使用强一致性, ACID):

MySQL, PostgreSQL, Oracle, SQL Server

02

NewSQL (强一致性):

Google Spanner,
CockroachDB,
TiDB

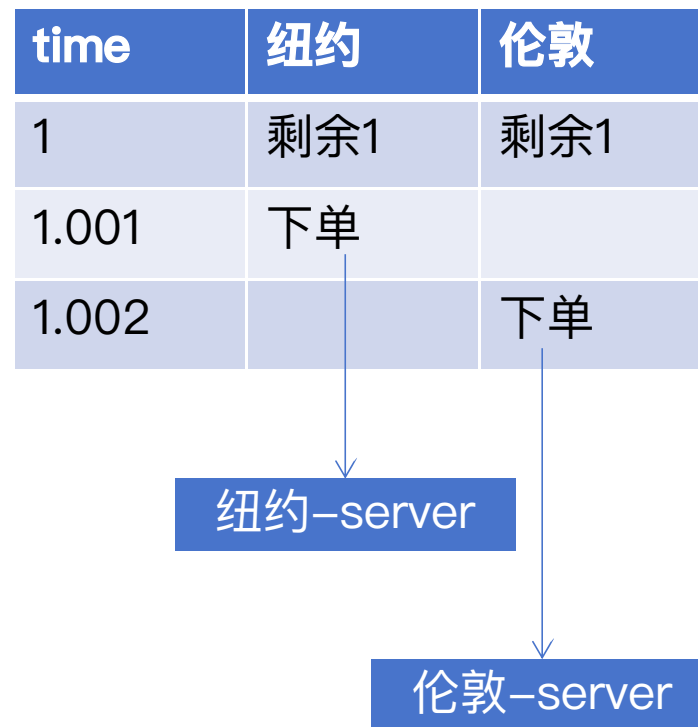
03

NoSQL (可配置或最终一致性):

DynamoDB (可配置),
Cassandra (最终),
MongoDB (副本集内强一致, 分片集群最终一致)

Google Spanner实现强一致性

- 场景：
 - 假设有一个全球知名的时尚品牌，限量销售 1000 双。
 - 官方电商平台的全球站点（例如，纽约、伦敦、东京）同时开售。
- 可能出现的问题：
 - 超卖，虚假库存下单失败，客户提交顺序不一致
- Google spanner的效果：
 - 库存准确，
 - 用户看到最新数据，
 - 操作的原子性和顺序



Google Spanner实现强一致性

• 关键技术

• TrueTime:

- 提供了一个具有已知不确定性范围的全局时间参考。
- 99 %情况下的宽度 < 1 毫秒。
- 10:00:00.123456 和 10:00:00.124000

• 事务提交时间戳:

- 利用 TrueTime 为每个事务分配一个全局有序的提交时间戳。

• 提交等待:

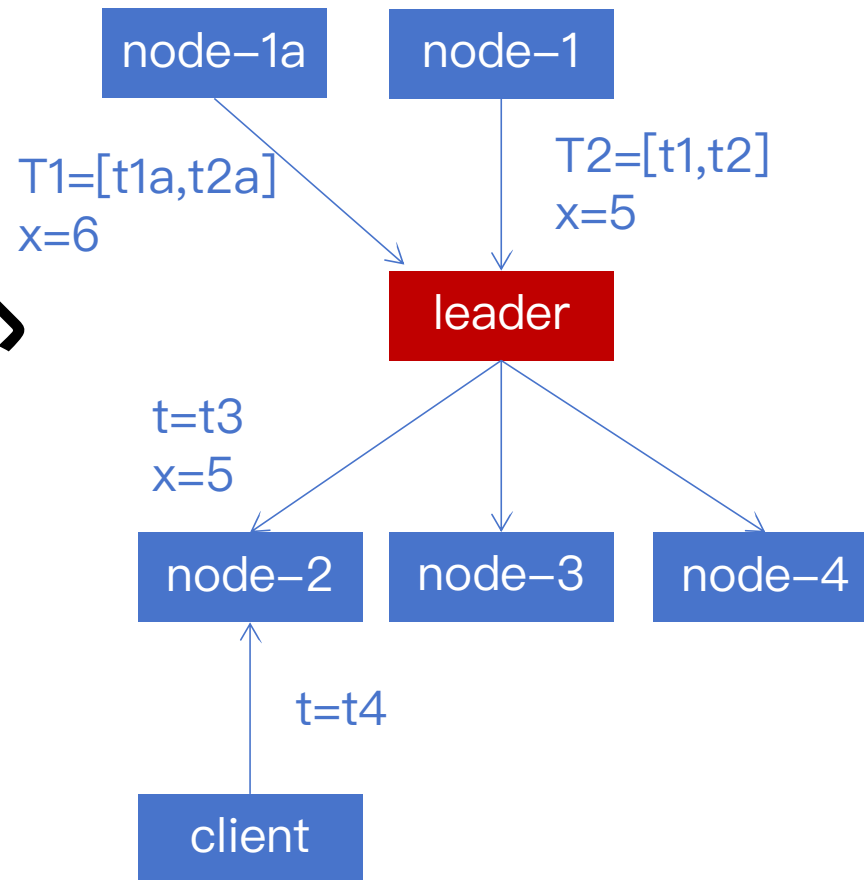
- $TT.now().earliest > ts1$ 才让事务成功。
- 事务成功后, 修改内容可见

• Paxos/Raft:

- 在每个数据分片内部保证数据复制和一致性。

• 两阶段提交 (2PC):

- 协调跨多个分片的事务的原子性。



强一致性的算法

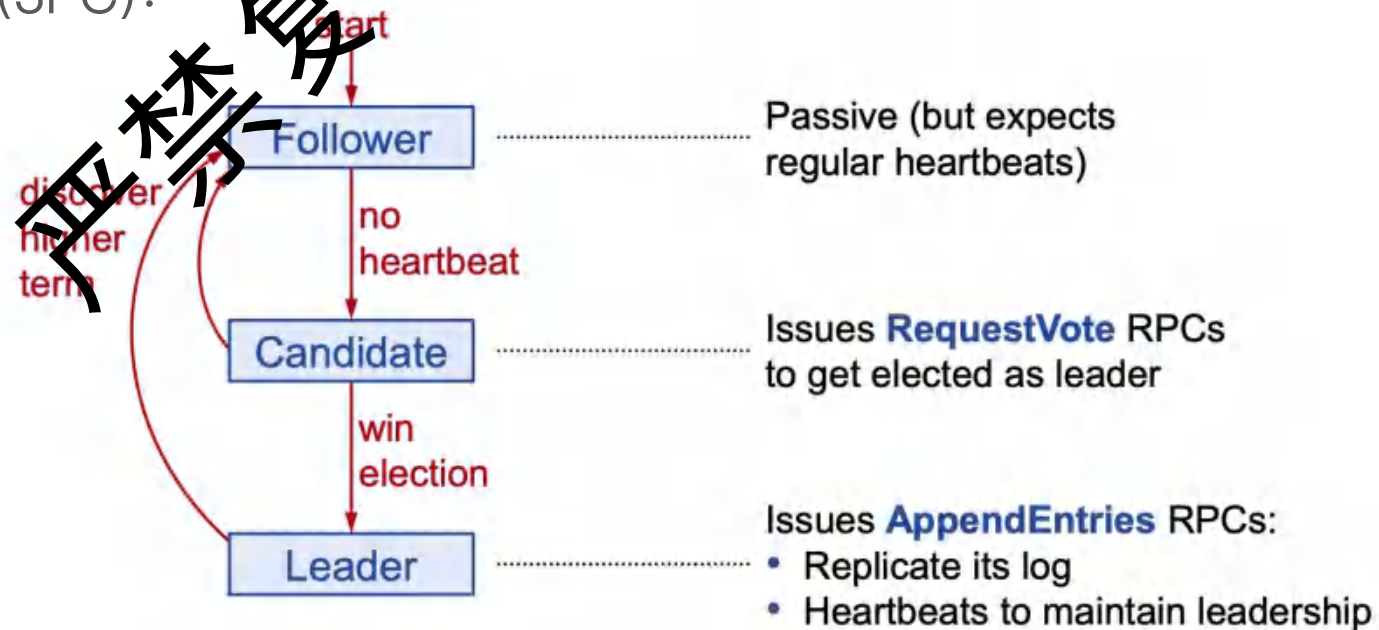
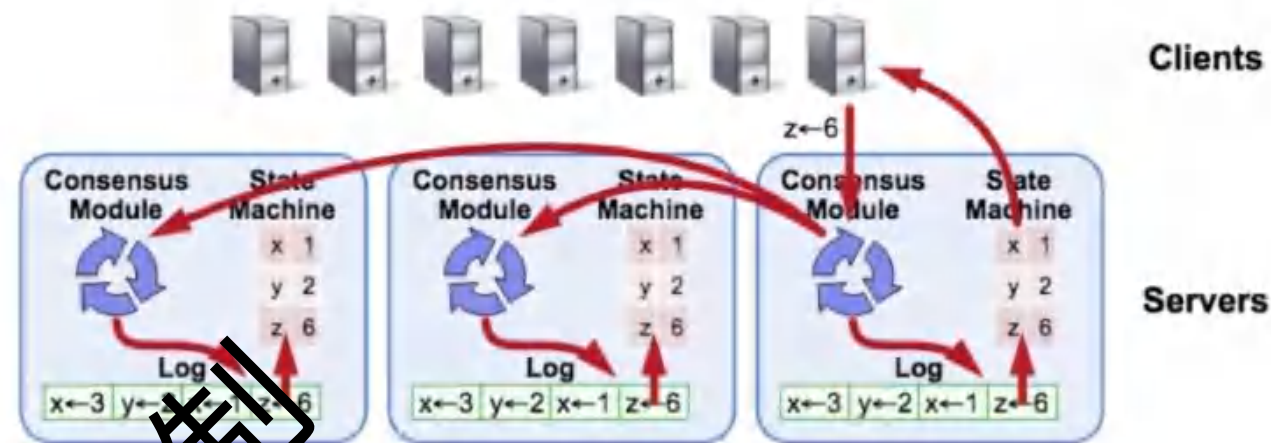
- 强一致性:

- 共识算法:

- Paxos,
- Raft (用于日志复制和状态机复制)

- 两阶段提交 (2PC) / 三阶段提交 (3PC):

- 用于分布式事务



<https://raft.github.io/slides/uiuc2016.pdf>

<https://raft.github.io/>

Cassandra实现最终一致性

- 业务场景：社交媒体帖子的点赞计数器。
 - 高并发写入：
 - 1000用户为同一个热门帖子点赞，“点赞数”的写入请求非常频繁。
 - 高可用性要求：
 - 用户希望点赞操作能够快速响应。
 - 对点赞数的容忍度：
 - 对于点赞数，用户通常可以容忍轻微的延迟或暂时的不一致。
- Cassandra的实现方式：
 - 写一致性级别：local_one（任一server写入）
 - 读一致性级别：local_one（任一server读出）
 - 最终一致性：
 - 用户在不同的server上看到的点赞数短期内可能不一样。
 - hinted handoff, read repair, nodetool repair。

最终一致性是怎么实现的

Quorum NRW

- N个副本，W个写成功，R个读成功， $W+R > N$ 保证强一致，否则可能最终一致。

Gossip协议:

- 节点间信息交换与同步。

版本向量 (Version Vectors)

- 检测 and 解决数据冲突。

Cassandra是如何使用Quorum NWR的

- NWR的定义：
 - N (Replication Factor 复制因子): 定义了每一份数据在集群中存储的副本数量。
 - W (Write Consistency Level 写一致性级别): 写操作成功之前，必须有多少个副本节点确认写入。
 - R (Read Consistency Level 读一致性级别): 读操作被认为成功之前，多少个副本节点读取数据并响应。
- 通过调整 W 和 R 的值，开发者可以在一致性、可用性和延迟之间进行权衡。
 - 如果 $W + R > N$ ，则可以保证强一致性（读取操作至少会看到最近一次写入操作的结果，因为读写的副本集合必然有重叠）。
 - 如果 $W + R \leq N$ ，则可能发生最终一致性，即读取操作可能不会立即看到最新的写入。
- Cassandra 每次读写操作都可以指定不同的
 - ONE (最弱), QUORUM (强一致性), LOCAL_QUORUM (多数据中心), EACH_QUORUM, ALL (全部)

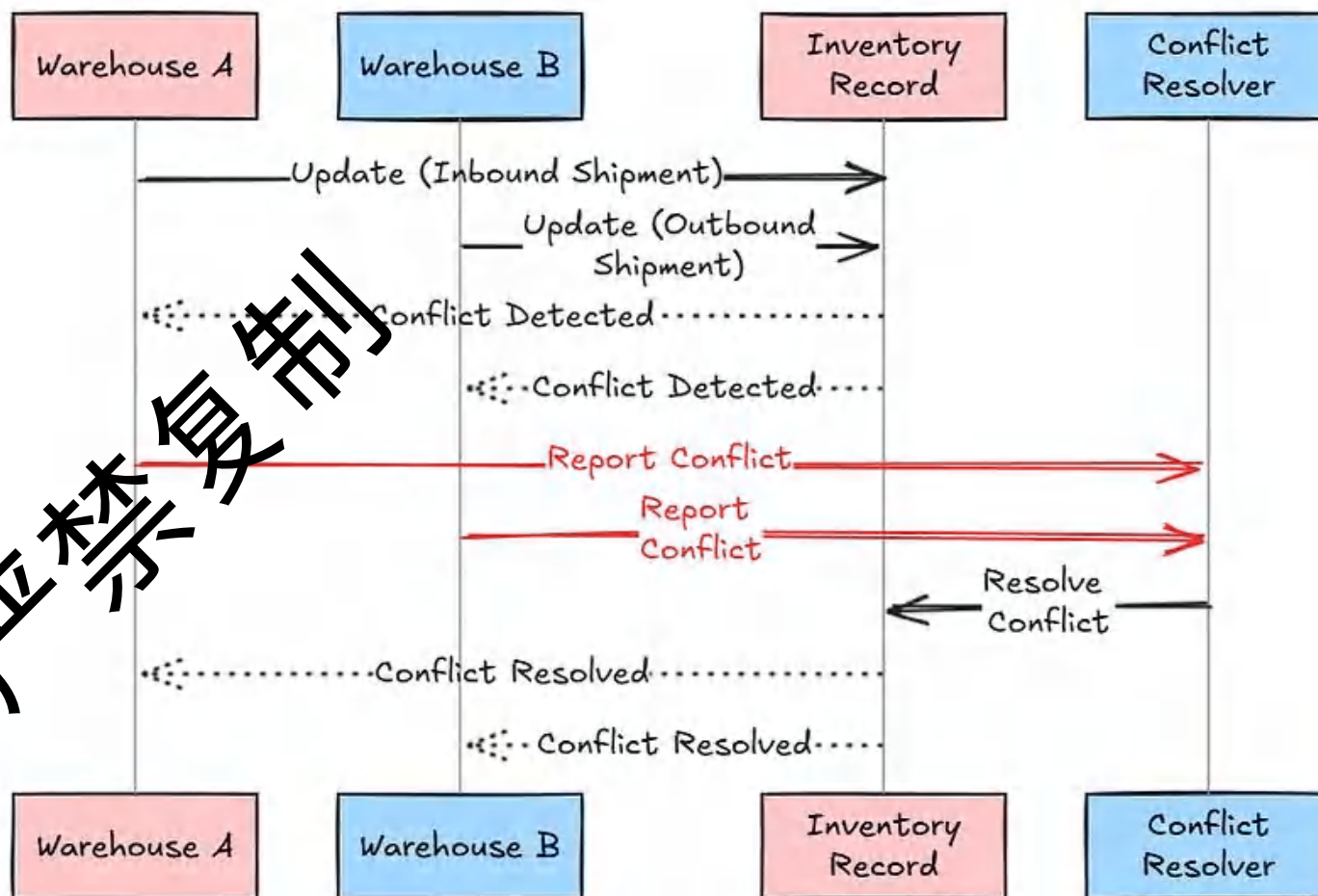
版本向量解决并发修改冲突

- 版本向量

- 每个节点维护一个版本向量 $[n_1c_1, n_2c_2, \dots]$
 - n_i 表示第几个节点
 - c_i 表示节点 i 的更新计数
- 节点执行本地更新时，增加自己对应的计数器值
- 节点间通信时交换版本向量并合并（通常取每个位置的最大值）

- 合并：

- 相等
- 兼容：全部小于或等于对方
- 冲突：部分小于等于，部分大于



<https://www.linkedin.com/in/shanoj-kv/>

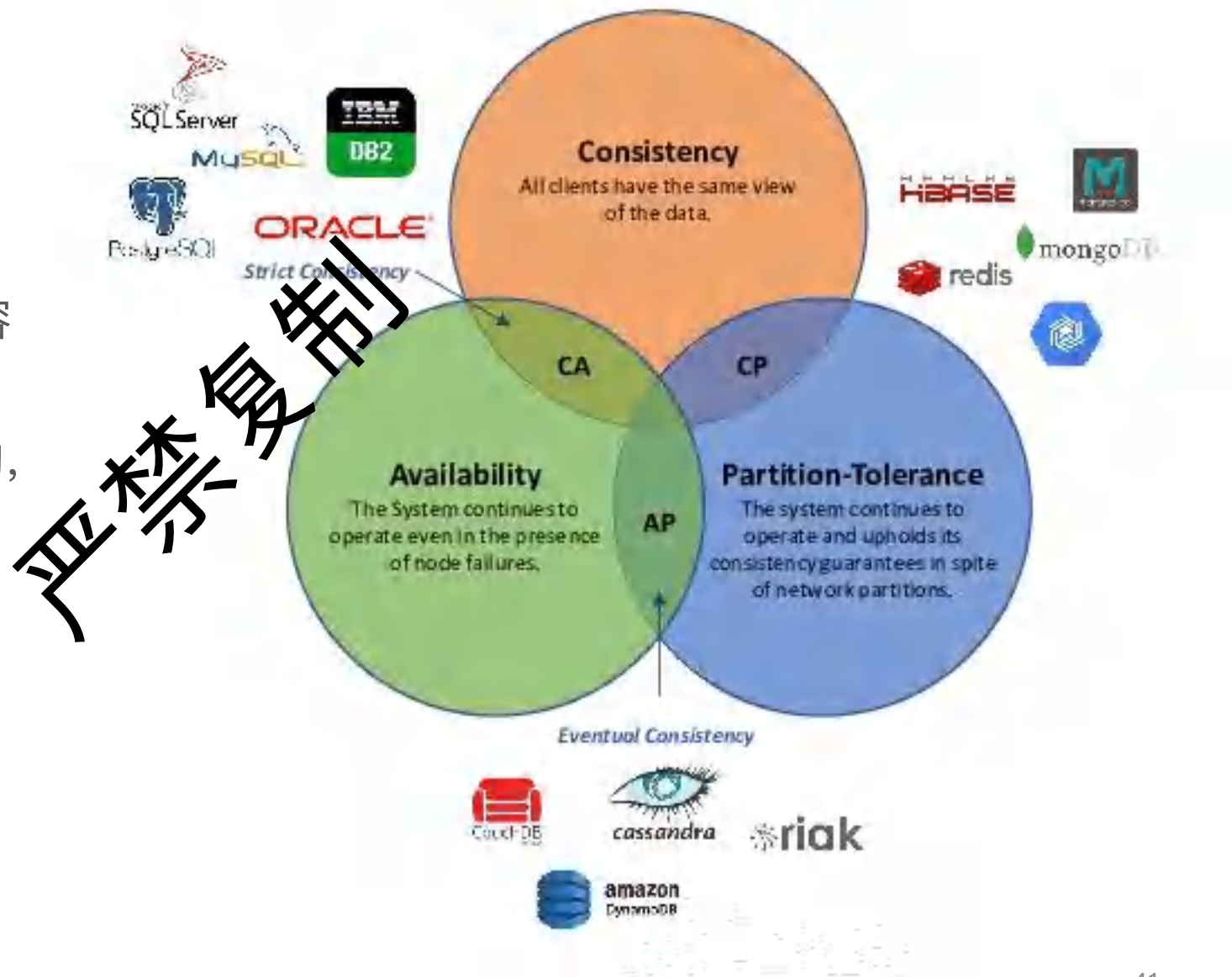


Shanoj

深化问题：CAP理论与BASE理论

- CAP理论:

- C (Consistency): 一致性
- A (Availability): 可用性
- P (Partition Tolerance): 分区容错性
- 在分布式系统中, P是必须保证的, C和A只能取其一。





03 复制数据读取性能

应用场景

- 定义：
 - 系统需要快速响应用户请求，
 - 或在单位时间内处理大量读取操作。
- 典型场景：
 - 快速响应 (低延迟):
 - 用户画像实时查询、
 - 实时推荐引擎、
 - 在线游戏状态读取。
 - 高吞吐 (高QPS/TPS):
 - 电商首页商品展示、
 - 新闻Feed流、
 - 热门排行榜。

严禁复制

进一步深化的问题



01

对延迟的具体要求是什么？

(平均延迟、P95/P99延迟)

02

对吞吐量的具体要求是什么？

(峰值QPS、日常QPS)

03

读操作的模式是怎样的？

(随机单点查询 vs. 范围扫描 vs. 聚合分析)

04

数据的新鲜度要求如何？

(是否可以接受缓存带来的轻微延迟)

05

读写比例是怎样的？

(读多写少 vs. 写多读少)

亚泰复生

相关数据库

内存数据库 (极低延迟):

Redis, Memcached

01

NoSQL (高吞吐):

键值存储: DynamoDB,
Redis

02

列式存储:

HBase, Cassandra (特定查询
模式下高效)

03

搜索引擎 (复杂查询与文本搜索):

Elasticsearch, Apache Solr

04

关系型数据库 (优化后):

通过索引、查询优化、读写分离

05

高吞吐：大型电商平台的“黑色星期五”

- 场景描述：

- 一家大型电商平台计划在“黑色星期五”当天举办一场大规模的闪购活动。在活动开始的特定时间点完成购买

- 挑战：

- 巨大的并发读取请求：
 - 商品详情页浏览：数百万用户同时刷新和浏览热门促销商品的详情页。
 - 实时库存查询：系统需要频繁查询实时库存，以避免超卖。
- 巨大的并发写入请求：
 - 订单创建：在短时间内有海量的订单被创建，写入数据库。
 - 库存扣减：商品的库存需要实时扣减。
 - 购物车更新：大量用户同时添加、删除购物车商品。

Dynamo如何支持高吞吐

- 可扩展的读取和写入容量：
 - 提高相关数据表（如商品表、库存表、订单表）的读/写取容量单位 (RCU/WCU) 。
- 高效的数据访问：
 - 主键查询，全局二级索引 (GSI)：
- 分区和水平扩展：
 - 自动将数据分区到多个存储节点
- 原子计数器 (Atomic Counters)
 - UpdateItem 原子性地增加或减少，高并发时保证数据一致性
- DAX (DynamoDB Accelerator) 可选优化：
 - DAX 作为缓存层。DAX 能够将读取延迟降低到微秒级别

关键技术实现简述

> 索引:

B-Tree/B+Tree, LSM-Tree (写优化, 读放大), 哈希索引, 倒排索引, 全文索引。

> 缓存:

客户端缓存, CDN, 代理缓存, 数据库内置缓存, 分布式缓存 (Redis, Memcached)。

> 数据分区/分片 (Sharding):

将数据分散到多个节点, 并行处理读取请求。

> 读写分离:

主库写, 从库读, 分散读取压力。

> 物化视图:

预计算结果, 加速查询。

LSM



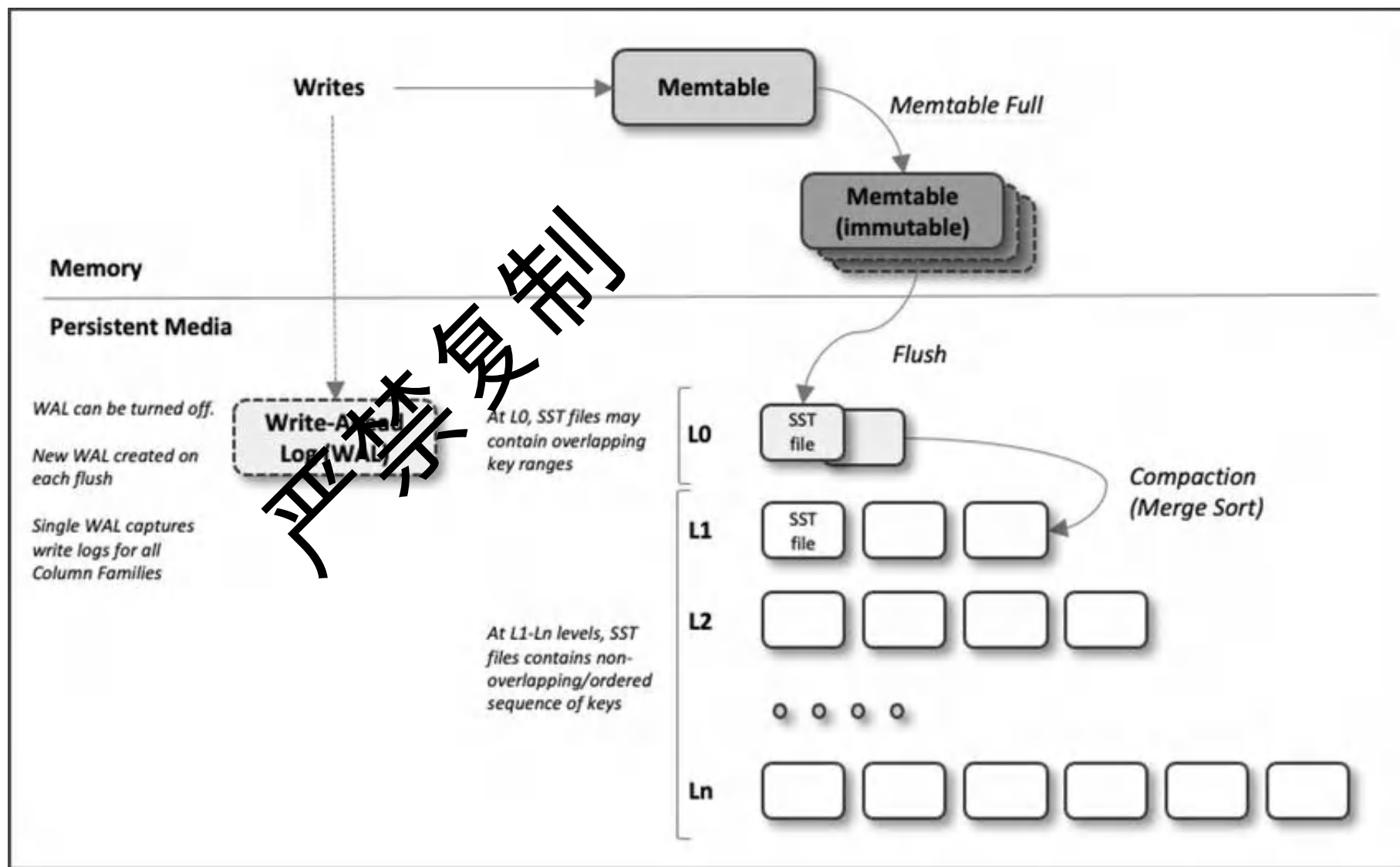
LSM

- 组成要素:

- 内存写入缓存memtable;
- 事务log;
- Sorted String Tables 磁盘上的有序文件
 - data
 - index
 - metadata
- 分层架构和合并
- 列族;

- 适用场景

- 写密集
- 倒排索引



RocksDB在Twitter的应用

- 在2017年，Twitter 的 Manhattan 团队对 RocksDB 进行了评估
 - 数据使用量减少约 50%：
 - 相比于 BTree 后端，RocksDB 提供了更好的数据压缩效果。
 - CPU 使用量减少约 20%：
 - 优化后的 RocksDB 后端在处理相同工作负载时，CPU 消耗更低。
 - 延迟相当或更优：
 - 在读写操作的延迟方面，RocksDB 表现与原有系统相当或更优的性能。

https://blog.x.com/engineering/en_us/topics/infrastructure/2021/adopting-rocksdb-within-manhattan

THANK YOU

北京交通大学软件学院

严禁复制

